Laboratorio 2 — Semáforos en XV6

Clase 2: carreras, semáforos y syscalls

Sistemas Operativos — FaMAF, UNC

17/09/2026

Agenda

- Condiciones de carrera
- Semáforos
- Syscalls en xv6
- Primer stub que compile

Condiciones de carrera

La pregunta que quedó de la clase 1

Si dos procesos quieren imprimir al mismo tiempo, ¿qué puede salir mal?

- Las líneas se mezclan en la salida.
- Peor: si comparten una variable sin protección, el resultado es impredecible.

Una línea inocente

```
contador += 1;
```

- Supongamos que dos procesos ejecutan esa línea sobre el mismo contador.
- Empieza en 5. Deberían dejarlo en 7.

No es una operación: son tres

```
contador += 1;     se traduce a:

    lee     contador   →  registro
    suma    registro + 1
    escribe registro   →  contador
```

- El scheduler puede interrumpir entre cualquiera de las tres.

El incremento perdido

Proceso	Qué hace	contador
—	estado inicial	**5**

El incremento perdido

Proceso	Qué hace	contador
—	estado inicial	**5**
A	lee contador, se lleva un 5	5

El incremento perdido

Proceso	Qué hace	contador
—	estado inicial	**5**
A	lee contador, se lleva un 5	5
	↓ *cambio de contexto* ↓	

El incremento perdido

Proceso	Qué hace	contador
—	estado inicial	**5**
A	lee contador, se lleva un 5	5
	↓ *cambio de contexto* ↓	
B	lee contador, se lleva un 5	5

El incremento perdido

Proceso	Qué hace	contador
—	estado inicial	**5**
A	lee contador, se lleva un 5	5
	↓ *cambio de contexto* ↓	
B	lee contador, se lleva un 5	5
B	escribe $5 + 1$	**6**

El incremento perdido

Proceso	Qué hace	contador
—	estado inicial	**5**
A	lee contador, se lleva un 5	5
	↓ *cambio de contexto* ↓	
B	lee contador, se lleva un 5	5
B	escribe $5+1$	**6**
	↓ *cambio de contexto* ↓	

El incremento perdido

Proceso	Qué hace	contador
—	estado inicial	**5**
A	lee contador, se lleva un 5	5
	↓ *cambio de contexto* ↓	
B	lee contador, se lleva un 5	5
B	escribe $5+1$	**6**
	↓ *cambio de contexto* ↓	
A	escribe $5+1$ (con su 5 viejo)	**6**

El incremento perdido

Proceso	Qué hace	contador
—	estado inicial	**5**
A	lee contador, se lleva un 5	5
	↓ *cambio de contexto* ↓	
B	lee contador, se lleva un 5	5
B	escribe $5+1$	**6**
	↓ *cambio de contexto* ↓	
A	escribe $5+1$ (con su 5 viejo)	**6**

Dos incrementos, un solo efecto.

El de A pisó al de B.

Condición de carrera

El resultado depende del orden en que el scheduler intercala los procesos.

- El código no cambió; cambió **cuándo** se ejecuta cada parte.
- Por eso no alcanza con leer el programa: hay que pensar en todos los entrelazados posibles.

En xv6

- Varios procesos corren en paralelo real: 3 CPUs (`-smp 3`).
- Comparten recursos del kernel: tablas, buffers, semáforos.
- El kernel debe garantizar **atomicidad** en las operaciones de sincronización.

El temporizador puede interrumpir a un proceso **en cualquier instrucción** — incluso en la mitad de un `contador++`.

Tres mecanismos que combinan los semáforos

Mecanismo	Idea
Exclusión mutua	Sólo un proceso a la vez en la sección crítica
Espera bloqueante	Si no hay recurso, el proceso duerme (no hace busy-wait)
Señalización	Un proceso avisa a otro que puede continuar

Primer intento: un candado

La idea más simple: una variable que diga libre u ocupado.

```
lock(&l);
    contador += 1;      // sección crítica
unlock(&l);
```

- Si sólo uno puede entrar a la vez, el incremento no se pierde.

¿Y si dos lo toman al mismo tiempo?

```
if (libre) {          ← A mira: libre
                      ← B mira: libre  (¡también!)
    libre = 0;        ← los dos entran
}
```

- El candado tiene la misma carrera que queríamos evitar.
- **Verificar y tomar** tiene que ser una sola operación indivisible: la da el hardware (en RISC-V, la instrucción `amoswap`).

El precio: busy-wait

Si el candado está tomado, el que espera gira quemando CPU:

```
while (test_and_set(&libre) == 0)
    ;               // no hace nada, sólo gasta ciclos
```

- Aceptable si la espera dura **nanosegundos**.
- Un desperdicio si hay que esperar a otro proceso, a un disco o a una tecla.

Dos herramientas, dos escalas

	spinlock	**semáforo**
Cómo espera	girando (quema CPU)	durmiendo (suelta la CPU)
Cuánto dura	nanosegundos	lo que haga falta
Quién lo usa	sólo el **kernel**	procesos de **usuario**

En este lab van a usar los dos

Un **spinlock** adentro del kernel para proteger la tabla de semáforos unas pocas instrucciones… y el **semáforo** que le exponen a los procesos, que puede bloquear todo el tiempo que haga falta.

Semáforos

La idea de Dijkstra (1965)

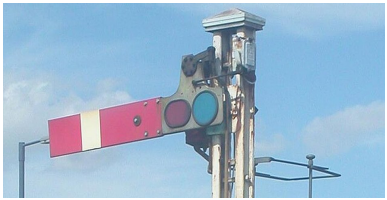
Un semáforo es un contador entero con dos operaciones atómicas:

P / wait / down — `sem_down`

Si el valor es > 0: lo decrementa.
Si es 0: **bloquea** al proceso.

V / signal / up — `sem_up`

Incrementa el valor.
Si hay procesos esperando, los **despierta**.



El de **trenes**, no el de la esquina.

Semáforo binario: un candado

```
sem_open(mutex, 1);

sem_down(mutex);
  // sección crítica — un solo proceso
sem_up(mutex);
```

Semáforo contador: fichas en una caja

- `sem_open(fichas, 3)` $\rightarrow$ 3 fichas disponibles.
- `sem_down` toma una ficha (espera si no hay).
- `sem_up` devuelve una ficha.

Un estacionamiento con 3 lugares: el 4.º auto espera afuera.

Vista previa: `pingpong`

Semáforo	Valor inicial	Quién lo usa
`turn_ping`	**1**	El padre imprime “ping”
`turn_pong`	**0**	El hijo imprime “pong”

(entrega intermedia — 28/09)

pingpong: el protocolo

```
padre: sem_down(ping) → printf ping → sem_up(pong)
hijo:  sem_down(pong) → printf pong → sem_up(ping)
```

- Cada proceso baja el suyo y sube el del otro.

Y más adelante: prodcons

- Un proceso produce ítems; otro los consume.
- Entre los dos, un buffer de capacidad **B**.
- Si el buffer se **llena**, el productor espera.
- Si se **vacía**, el consumidor espera.

Semáforo	Arranca en	Cuenta
`empty`	**B**	lugares libres en el buffer
`full`	**0**	ítems listos para consumir

Lo vemos en la clase 6. Primero, `pingpong`.

Semáforos nombrados: las reglas del lab

Propiedad	**Política**
Administrador	**El kernel**
Alcance	**Global** — todos los procesos
Nombre (id)	Entero **1..64** (el 0 es inválido)
Persistencia	Mientras xv6 esté encendido
Retorno	**0** = error, **id** $>$ **0** = éxito

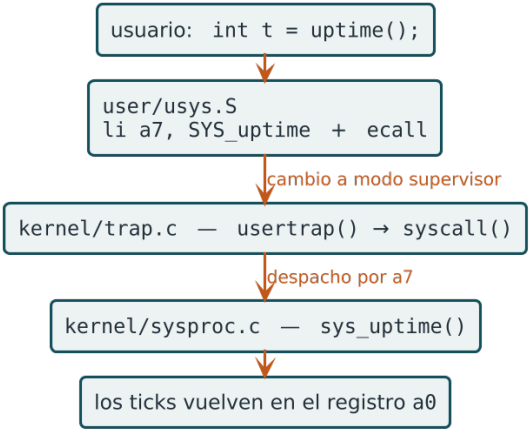
Políticas obligatorias (no las inventan ustedes)

- `sem_up` puede incrementar más allá del valor inicial — es un contador real, no un binario forzado.
- `sem_close` está permitido con procesos bloqueados: esos `sem_down` deben retornar **0** al despertar.
- Valor máximo: `SEM_VALUE_MAX = 1 000 000`.

Detalle completo en el enunciado, sección *Políticas obligatorias*.

Syscalls en xv6

Una syscall que ya existe: `uptime()`



Del otro lado: sys_uptime

```
uint64
sys_uptime(void)
{
  uint xticks;

  acquire(&tickslock);
  xticks = ticks;
  release(&tickslock);

  return xticks;
}
```

- Devuelve los **ticks** desde que arrancó xv6 (uno cada $\sim 1/10$ s, los cuenta el temporizador).
- Toma un spinlock para leer `ticks`: es una variable compartida entre las 3 CPUs.

Los 6 lugares donde agregan sem_*

Capa	Archivo	Qué agregan
User API	`user/user.h`	`int sem_open(int, int);`
Stub asm	`user/usys.pl`	`entry("sem_open");`
Números	`kernel/syscall.h`	`#define SYS_sem_open 22`
Tabla	`kernel/syscall.c`	una entrada en `syscalls[]`
Wrapper	`kernel/sysproc.c`	`sys_sem_open()`: lee args
Lógica	`kernel/semaphore.c`	tabla, `down`/`up`, sleep/wakeup

El wrapper en sysproc.c

```
uint64
sys_sem_open(void)
{
  int id, value;

  argint(0, &id);
  argint(1, &value);

  return sem_open(id, value);   // 0 = error, id > 0 = ok
}
```

Ojo

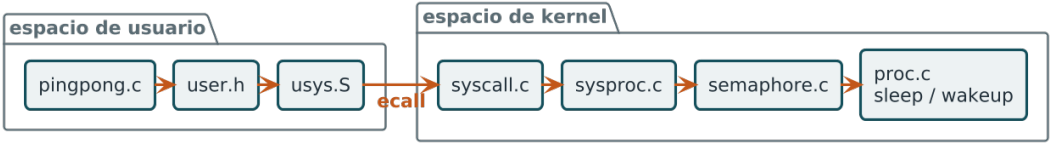
En este xv6 `argint` devuelve `void`: no se chequea su retorno. La validación del `id` va **adentro** de `sem_open`.

Sleep y wakeup (lo vemos en la clase 3)

- Cuando `sem_down` encuentra el valor en 0, el proceso duerme en un canal y otro proceso sigue corriendo.
- `sem_up` lo despierta.
- Se implementa con `sleep()` y `wakeup()` de `kernel/proc.c`.

Dormir no es lo mismo que esperar girando: el proceso **suelta la CPU**.

El camino completo de una sem_*



Mapa de archivos y tests

Qué archivos se tocan

```
kernel/
  semaphore.c    ← implementación principal
  semaphore.h
  defs.h, param.h, main.c
  sysproc.c, syscall.c, syscall.h

user/
  user.h, usys.pl
  pingpong.c     ← ustedes (parcial)
  prodcons.c     ← ustedes (final)
  semtest.c      ← kickstarter (no modificar)
```

Y el Makefile

```
UPROGS += $U/_pingpong $U/_prodcons
OBJS   += $K/semaphore.o
```

- Si no está en `UPROGS`, el programa no entra en la imagen y no lo pueden correr desde el shell.

Auto-corrección

Comando	Entrega	Score
`make grade-parcial`	**28/09**	**11/11**
`make grade-final`	**07/10**	**15/15**

Bloque	Tests de la parcial
`semtest` (6 pts)	noinit, init, closeuse, exhaust, grow, reopen
`pingpong` (5 pts)	args, orden, concurrencia

Primer stub que compile

Checklist mínimo

- ☐ Crear `kernel/semaphore.c` con los stubs.
- ☐ Registrar las 4 syscalls (los 6 lugares).
- ☐ Llamar `seminit()` desde `main.c`.
- ☐ `make qemu` compila y entra al shell.

Los stubs

```
// kernel/semaphore.c — TEMPORAL (se reemplaza en la clase 3)
int sem_open(int id, int value) { return 0; }
int sem_close(int id)           { return 0; }
int sem_up(int id)              { return 0; }
int sem_down(int id)            { return 0; }
```

Para la casa

1. Repo compilando con los stubs.
2. Leer `kernel/proc.c`: `sleep()` y `wakeup()`.
3. Leer el enunciado: políticas + sección `pingpong`.

Entrega intermedia: **28/09 23:59**

Referencias

- **OSTEP** — Semáforos (cap. 31) y Locks (cap. 28).
- **xv6 book** — cap. 2 (system calls), cap. 7 (scheduling).
- **Enunciado** — *Implementación en el kernel*.